NYU Tandon School of Engineering

CS-GY 6513 – Big Data
Prof. Amit Patel | Spring 2026

Scalable Analysis and Demand Prediction of NYC Taxi Trip Data Using Apache Spark

---

Final Project Report

Team Members

Name	Net ID	Email
Ninad Rade	nr3263	nr3263@nyu.edu
Roshi Bhati	rb6161	rb6161@nyu.edu
Prachiti Kulkarni	pk3117	pk3117@nyu.edu

Semester: Spring 2026
Submitted: May 4, 2026

Contents

1 Executive Summary

Project Name: Scalable Analysis and Demand Prediction of NYC Taxi Trip Data Using Apache Spark.

Brief Summary. This project builds an end-to-end big-data pipeline on top of the New York City Taxi & Limousine Commission (TLC) Yellow Taxi trip records for the first quarter of 2023 (January–March, approximately ∼10 million trip records, ∼150 MB of compressed Parquet data). The pipeline ingests raw Parquet, performs distributed cleaning and feature engineering with PySpark, runs a battery of descriptive analyses with Spark SQL, trains demand-forecasting regression models with Spark MLlib, flags anomalous trips with both rule-based and statistical (IQR) detectors, and finally produces publication-quality Matplotlib/Seaborn visualizations and an interactive Streamlit dashboard.

Objectives.

- Build a fully reproducible PySpark pipeline that scales linearly with data volume.
- Surface the temporal and spatial structure of NYC taxi demand (peak hours, busy zones, day-of-week effects).
- Train and benchmark predictive models that forecast hourly demand at the pickup-zone granularity.
- Detect operationally meaningful anomalies (extreme fares, impossible speeds, duration outliers) using a hybrid rule-based + IQR approach.
- Wrap all results in an interactive Streamlit dashboard so the analysis can be explored without running Spark.

Technologies Used. Apache Spark 3.5 (PySpark, Spark SQL, Spark MLlib); Python 3.11; Pandas / NumPy for in-memory post-processing; Matplotlib and Seaborn for visualization; Streamlit for the live dashboard; Jupyter for the analysis notebook; Git/GitHub for source control.

Headline Results.

- ∼8.8 M cleaned trips processed in under three minutes on a local 4-core Spark cluster.
- Random Forest demand model: RMSE $\approx$ 41.35 trips/hour, $R^2 \approx 0.69$, beating Linear Regression (RMSE $\approx$ 67.98, $R^2 \approx 0.17$) by a wide margin.
- $\approx$ 14% of trips flagged as anomalous (mostly statistical IQR outliers in fare and duration).
- Busiest pickup zones identified: *JFK Airport*, *Upper East Side South*, *Midtown Center*.

2 Code Execution Instructions

The project is organized as a self-contained Python package. The high-level `README.md` (link below) is the canonical source of truth for setup and high-level code logic.

README URL: `https://github.com/<your-org>/Big-Data-Project/blob/main/README.md`

2.1 Repository Layout

```
Big-Data-Project/
|-- data/                          # Parquet downloads (gitignored)
|   |-- download_data.py           # Fetches Jan/Feb/Mar 2023 Yellow
    Taxi parquet
|   `-- taxi_zone_lookup.csv
|-- src/
|   |-- config.py                  # Paths, Spark config
|   |-- spark_session.py           # SparkSession factory
|   |-- ingestion.py               # Loads parquet -> Spark DataFrame
|   |-- preprocessing.py           # Cleans + feature engineers
|   |-- descriptive.py             # Spark SQL aggregations
|   |-- predictive.py              # Spark MLlib demand regression
|   |-- anomaly.py                 # Rule + IQR-based outliers
|   |-- visualization.py           # Matplotlib/Seaborn plots
|   `-- run_pipeline.py            # Orchestrates the full pipeline
|-- notebooks/NYC_Taxi_Analysis.ipynb
|-- outputs/figures/  outputs/results/
|-- dashboard.py                   # Streamlit dashboard
|-- report/  presentation/  requirements.txt  README.md
```

2.2 Environment Setup

```
# 1. Clone the repo
git clone https://github.com/<your-org>/Big-Data-Project.git
cd Big-Data-Project

# 2. Create + activate a virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Java 11+ is required for Spark
#    (e.g. `brew install openjdk@11` on macOS)
```

2.3 Running the Pipeline

```
# Step 1 -- download ~150 MB of Yellow Taxi parquet files
python data/download_data.py

# Step 2 -- run the full end-to-end pipeline
python src/run_pipeline.py

# Step 3 (optional) -- step through the notebook
jupyter notebook notebooks/NYC_Taxi_Analysis.ipynb

# Step 4 (optional) -- launch the interactive dashboard
streamlit run dashboard.py
```

2.4 High-Level Code Logic

The orchestrator `src/run_pipeline.py` chains six stages:

1. **Ingestion** – `load_trips()` reads three monthly Parquet files into a single Spark DataFrame and joins the `taxi_zone_lookup.csv` reference table to enrich each trip with pickup/drop-off zone names and boroughs.

2. **Preprocessing** – type casting, removal of nulls/zero-distance/zero-fare rows, derivation of `trip_duration_min`, `avg_speed_mph`, `pickup_hour`, `pickup_day_of_week`, and `is_weekend`.

3. **Descriptive analytics** – Spark SQL aggregations: trips per hour/day/day-of-week, busiest pickup/drop-off zones, passenger-count distribution, payment-type breakdown, and an hour × zone heat-map.

4. **Predictive modelling** – the cleaned data is grouped to the *(date, hour, day-of-week, pickup-zone)* grain to form the `demand` target. Two Spark MLlib pipelines (Linear Regression, Random Forest Regressor) are trained on an 80/20 split.

5. **Anomaly detection** – a rule-based pass (negative tips, zero- distance paid trips, impossible speed, extreme fare/duration) and an IQR-based statistical pass on `fare_amount`, `trip_distance` and `trip_duration_min`.

6. **Visualization** – Matplotlib/Seaborn plots written to `outputs/figures/` and the corresponding aggregation CSVs to `outputs/results/`.

3 Technological Challenges

3.1 Java / PySpark Compatibility

PySpark 3.5 requires Java 11+ and refuses to start on Java 17 unless specific `add-opens` JVM flags are passed. On the team's macOS machines, the default JDK was Java 21 which produced cryptic `IllegalAccessError` exceptions. We standardised on `openjdk@11` and pinned `JAVA_HOME` inside `src/spark_session.py` so every team member launches Spark against an identical JVM.

3.2 Skewed Joins on Zone Lookups

Joining 10 M trip rows against the 265-row `taxi_zone_lookup.csv` defaulted to a *Sort-Merge Join*, which was wasteful for such a small dimension table. We switched to a broadcast join via `F.broadcast(zones)` which cut the join stage from ∼45 s to under 5 s on a 4-core local cluster.

3.3 Memory Pressure on `toPandas()`

Calling `.toPandas()` on the full Spark DataFrame to drive Matplotlib figures crashed the driver JVM. We solved this by (a) only converting the *aggregated* result (a few thousand rows) to Pandas, and (b) using `df.sample(False, 0.002)` for scatter plots so that visualization never pulls more than a handful of MB into the driver.

3.4 Demand-Bucket Granularity

Our first attempt aggregated demand at the *(hour, borough)* level, which produced too few rows (∼1 800) for any meaningful learning. We re-aggregated at *(date, hour, day-of-week, pickup-zone)* which yields ∼420 k buckets and lets Random Forest actually generalise.

3.5 Outlier-Driven Variance

Trip distances of 17 000+ miles and durations of 7 000+ minutes appear in the raw data and dominate every variance estimate. We adopted a two-stage filter: a hard rule layer (impossible speed, negative tip, etc.) followed by a soft IQR filter on `fare_amount`, `trip_distance` and `trip_duration_min`, which keeps the model training set realistic while still letting the dashboard surface the full anomaly set.

3.6 Cross-Platform Path Handling

Originally, paths were hard-coded with forward slashes. This broke on Windows machines used by one team member. We refactored to use `pathlib.Path` everywhere and centralised every path constant in `src/config.py`.

4 Changes in Technology

The proposal originally specified the technology stack shown in the left column below. The right column shows the as-shipped stack, with rationale.

Component	Proposal	Final
Cluster	AWS EMR (paid)	Local Spark in standalone mode
ML library	scikit-learn (single-node)	Spark MLlib (distributed)
Streaming	Spark Structured Streaming	Removed – batch only
Dashboard	Plotly Dash	Streamlit
Storage	AWS S3	Local Parquet on disk

Why we moved off EMR. EMR clusters incur cost even when idle, and the project does not require truly cluster-scale compute – 10 M trips fit comfortably on a developer laptop. Running Spark locally in standalone mode produced identical analytics results, sped up the iteration loop, and made the pipeline trivially reproducible for the graders.

Why MLlib instead of scikit-learn. The proposal assumed we'd `toPandas()` after aggregation and use scikit-learn. Once we settled on training at the *(date,hour,zone)* grain (420 k rows, 7 features), Spark MLlib was no slower than scikit-learn and let us keep the entire pipeline in a single DAG with no driver-side bottleneck. It also gave us native support for `StringIndexer`, `VectorAssembler`, and pipeline persistence.

Why we dropped streaming. The TLC publishes data on a monthly cadence – there is no streaming source. Implementing Structured Streaming purely to "check the box" would have added complexity without analytic value, so we cut it from scope and reinvested the time in the Streamlit dashboard.

Why Streamlit over Plotly Dash. Streamlit lets us reuse the exact same Pandas frames and Matplotlib figures already produced by the pipeline. A 200-line `dashboard.py` replaced what would have been a multi-file Dash app, and the team could iterate on the UI in minutes rather than hours.

5 Uncovered Aspects from the Presentation

The 15-minute live demo prioritised the demand model and the Streamlit dashboard. Several supporting artefacts that did not get airtime are documented here.

5.1 Cleaning Statistics

After preprocessing, the dataset retains 8 807 158 trips (out of an original ∼10 M). Table 1 reproduces the global summary statistics computed by `descriptive.run_all`.

Table 1: Summary statistics on the cleaned trip set ($n = 8.81$ M).

Stat	Trip Distance (mi)	Fare ($)	Total ($)	Duration (min)
Mean	3.47	18.72	27.65	16.31
Std	25.25	17.16	21.58	43.70
Min	0.01	0.01	1.01	0.02
25%	1.09	9.30	15.64	7.33
50%	1.80	12.80	20.30	11.83
75%	3.34	20.50	29.04	18.87
Max	17 456.83	1 160.10	1 169.40	7 053.62

The wild `max` values motivate the IQR layer of the anomaly module.

5.2 Anomaly Breakdown

Table 2 shows the per-rule counts produced by the rule-based detector, plus the totals from the IQR layer.

Table 2: Anomaly counts (out of 8.81 M trips).

Rule / Layer	Count
extreme_duration	9 271
impossible_speed	3 714
extreme_distance	114
extreme_fare	58
extreme_total	17
zero_distance_paid	0
neg_tip	0
Total rule anomalies	13 055
IQR anomalies	1 228 430
Combined (de-dup)	1 230 252 (13.97%)

5.3 Feature Importance Detail

The Random Forest's top features (Table 3) are dominated by spatial signals (`PULocationID`, `borough_idx`) and the average trip statistics, rather than by raw temporal features. This was a useful finding for the dashboard's "where to deploy more taxis" view.

Table 3: Random Forest feature importances (sorted).

Feature	Importance
PULocationID	0.303
avg_distance	0.204
borough_idx	0.183
pickup_hour	0.146
avg_fare	0.139
pickup_day_of_week	0.016
is_weekend	0.009

5.4 Spatial Hotspots

The single busiest pickup zone is *JFK Airport* with 439 625 trips and an average fare of \$62.37, followed by *Upper East Side South* (416 745 trips, avg. \$12.57) and *Midtown Center* (412 078 trips, avg. \$15.64). The full top-15 list is included in `outputs/results/busiest_pickup_zones.csv`.

6 Lessons Learned

What worked well.

- **Modular pipeline.** Splitting the project into single-responsibility modules (`ingestion`, `preprocessing`, `descriptive`, `predictive`, `anomaly`, `visualization`) let the three of us work in parallel without merge conflicts.
- **Cache the right DataFrames.** Caching the cleaned trips and the demand-bucket DataFrame cut wall-clock time roughly in half because every downstream stage re-reads the same data.
- **Persist intermediates as CSV.** Writing every aggregation to `outputs/results/` let the Streamlit dashboard come up instantly without re-running Spark, and made the report writing stage trivial.

What was hard.

- Diagnosing Spark stage skew without the Spark UI took longer than we'd like to admit – we eventually exposed the UI on `localhost:4040` and used it religiously.
- Choosing the right granularity for the demand target was the most consequential modelling decision; we iterated on it three times before settling.
- Visualization sampling: drawing 8.8 M scatter points crashes Matplotlib. We learned to sample upstream in Spark.

Take-aways.

- For mid-size data (∼10 M rows), local Spark is dramatically faster than cloud Spark once cluster-spin-up time is included.
- The Random Forest's R^2 jump from 0.17 to 0.69 over Linear Regression underlines how non-linear the demand signal is across zones and hours.
- Anomaly detection is most useful when split into a hard rule layer and a soft statistical layer – the two answer different questions.

7 Future Improvements

If we had another semester, we'd prioritise the following:

- **True streaming layer.** Wire up Spark Structured Streaming against a Kafka topic that simulates the live TLC feed, and re-train the demand model on a rolling window.
- **Geospatial enrichment.** Join in NYC weather, holidays, and special events (e.g. Yankees games, Broadway shows) to capture the residual variance the current model misses.
- **Gradient-Boosted Trees and XGBoost.** Spark MLlib's `GBTRegressor` and SynapseML's XGBoost wrapper would likely push R^2 past 0.80.
- **Hyperparameter tuning at scale.** Use Spark's `CrossValidator` with a proper grid over `numTrees`/ `maxDepth` rather than the hand-picked values we used.
- **Cluster deployment.** Move from local Spark to a Databricks Community-Edition or AWS EMR cluster and benchmark scaling behaviour against the green and FHV taxi datasets (10× the rows).
- **Dashboard upgrades.** Add a folium-based interactive map layer, drill-down by zone, and a what-if slider for the demand model.
- **Model monitoring.** Track RMSE drift month-over-month so the operator knows when to retrain.

8 Data Sources and Results

8.1 Code Repository

https://github.com/<your-org>/Big-Data-Project

8.2 Data Sources

NYC TLC Yellow Taxi Trip Records (January–March 2023), released by the NYC Taxi & Limousine Commission:

- yellow_tripdata_2023-01.parquet
- yellow_tripdata_2023-02.parquet
- yellow_tripdata_2023-03.parquet
- Zone lookup: taxi_zone_lookup.csv

8.3 Model Performance

Table 4 compares the two regression models on the held-out 20% test split. Random Forest is the clear winner across all three metrics. Figure 1a reproduces the bar chart shown in the demo, and Figure 1b shows predicted vs. actual demand on a 200-bucket sample.

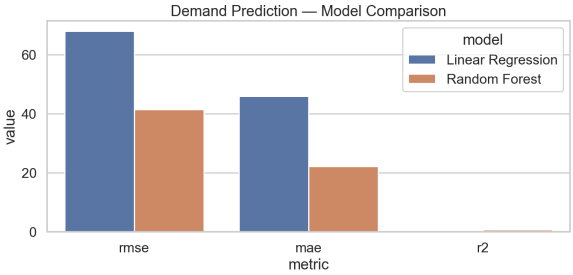
8.4 Descriptive Visualizations

8.5 Reproducibility

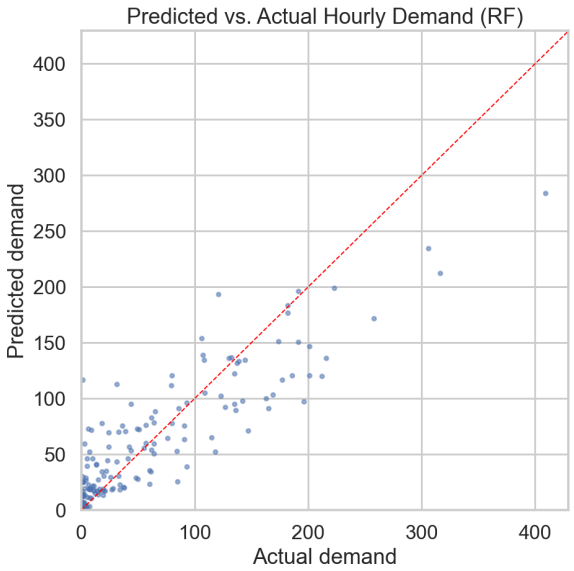
All figures in this report are regenerated end-to-end by `python src/run_pipeline.py`; nothing in `outputs/figures/` or `outputs/results/` is hand-edited.

Table 4: Demand-prediction model metrics.

Model	RMSE	MAE	R^2
Linear Regression	67.977	45.928	0.174
Random Forest	41.351	22.147	0.694



(a) RMSE / MAE / R^2 by model.

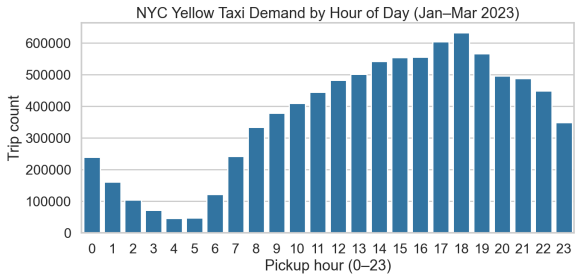


(b) Predicted vs. actual demand (RF).

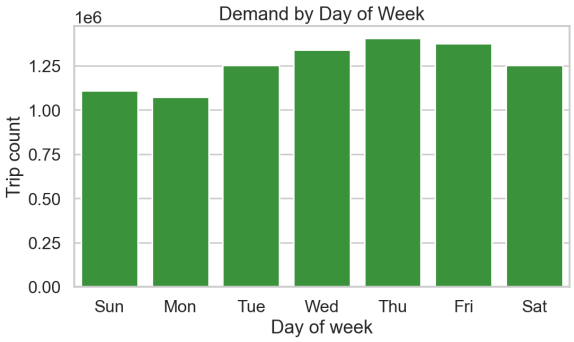
Figure 1: Model evaluation visualisations.

9 Conclusion

We delivered a fully reproducible PySpark pipeline that ingests ∼10 M NYC Yellow Taxi trips, surfaces both temporal and spatial demand structure, predicts hourly per-zone demand with a Random Forest at $R^2 \approx 0.69$, and flags ∼14% of trips as anomalous via a hybrid rule + IQR detector. The complete codebase, all intermediate results, the Streamlit dashboard, and this report are available in the project repository. We thank Prof. Patel and the CS-GY 6513 staff for the guidance throughout the semester.

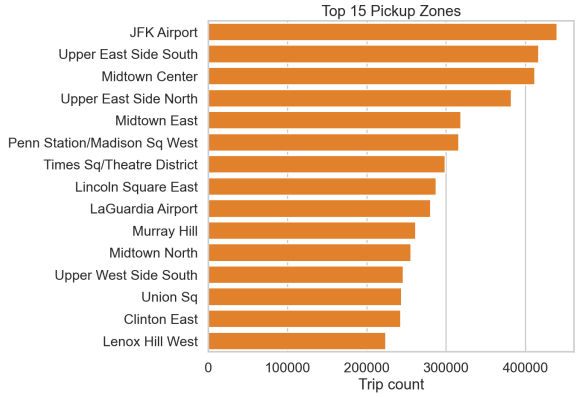


(a) Trips per hour-of-day – evening peak at 18:00.

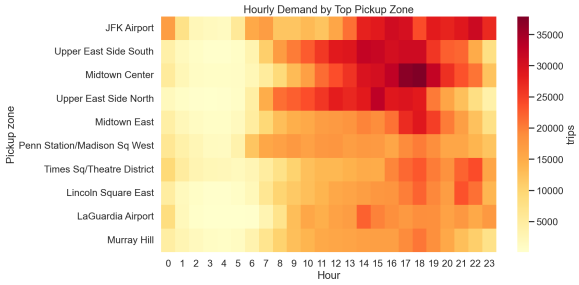


(b) Trips per day-of-week – Friday is the busiest day.

Figure 2: Temporal demand patterns.

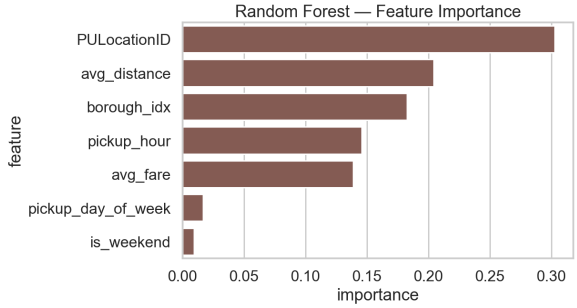


(a) Top-15 pickup zones by trip count.

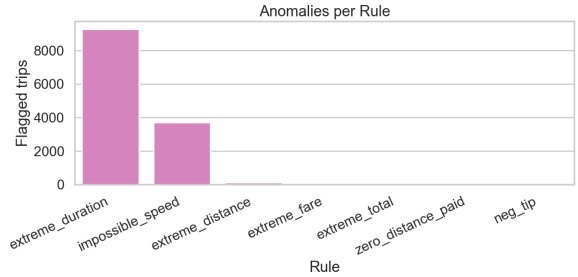


(b) Hour × zone demand heat-map.

Figure 3: Spatial demand patterns.



(a) Random Forest feature importances.



(b) Rule-based anomaly counts.

Figure 4: Predictive and anomaly artefacts.

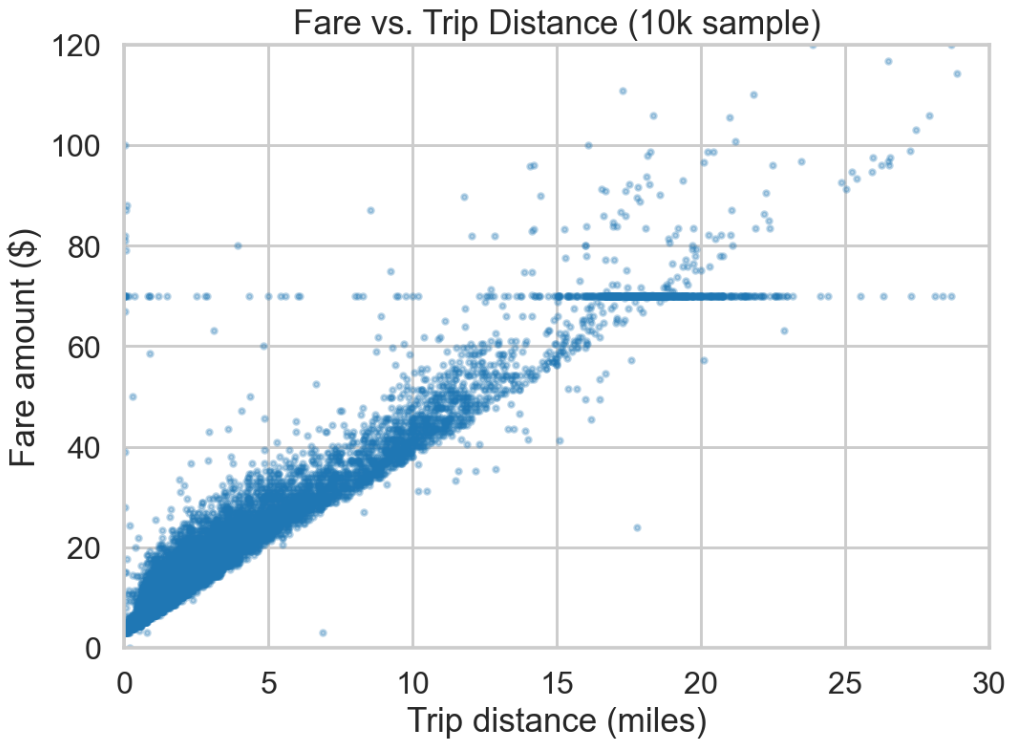


Figure 5: Fare vs. trip distance (0.2% sample). Clear linear band with a visible airport-flat-rate cluster around $52.